

Guia Completo: Implementando Kafka com .NET & Docker (EDA)

Este guia prático contém todo o passo a passo para configurar uma infraestrutura Kafka local via Docker e integrá-la com uma API .NET (Producer + Consumer) na arquitetura orientada a eventos.

1. Pacotes NuGet Necessários

No projeto C# (.csproj), adicione o pacote oficial da Confluent para suporte ao Kafka:

- **Nome do Pacote:** Confluent.Kafka
- **Comando via Terminal (PowerShell no diretório do projeto):**

PowerShell

```
dotnet add package Confluent.Kafka
```

2. Infraestrutura (Docker Compose)

Crie um arquivo chamado docker-compose.yml na raiz da solução do seu projeto e cole a estrutura abaixo (ajuste as portas de acordo com as disponíveis):

```
name: treino-kafka-estudos services: kafka: image: confluentinc/cp-kafka:7.6.0 container_name: kafka-
kraft-estudos ports: - "9092:9092" environment: - KAFKA_NODE_ID=1 -
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTE
XT_HOST:PLAINTEXT -
KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092 -
KAFKA_LISTENERS=PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092,CONTROLLER://0.0.0.0
:29093 - KAFKA_INTER_BROKER_LISTENER_NAME=PLAINTEXT -
KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER - KAFKA_PROCESS_ROLES=broker,controller -
KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:29093 -
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 - KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS=0
- KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=1 -
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1 - KAFKA_LOG_DIRS=/tmp/kraft-combined-
logs - CLUSTER_ID=MkU3OEVBNTcwNTJENDM2Qk kafka-ui: image: provectuslabs/kafka-ui:latest
container_name: kafka-ui-estudos ports: - "8095:8080" environment: -
KAFKA_CLUSTERS_0_NAME=local-cluster - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:29092 -
DYNAMIC_CONFIG_ENABLED=true depends_on: - kafka
```

Comandos de Execução no Terminal:

PowerShell

```
# Iniciar o ambiente Docker
docker compose up -d

# Para parar e resetar volumes se necessário
docker compose down -v
```

- Interface de Gerenciamento do Kafka: <http://localhost:8095>

3. Implementação da Aplicação em .NET

A. Classe do Consumidor (OrderCreatedConsumer.cs)

Crie o arquivo na pasta do projeto para atuar como o **Worker/Consumer**:

```
using System.Text.Json;
```

```
using Confluent.Kafka;
```

```
public class OrderCreatedConsumer : BackgroundService
```

```
{
```

```
    private readonly ILogger<OrderCreatedConsumer> _logger;
```

```
    private readonly ConsumerConfig _consumerConfig;
```

```
    public OrderCreatedConsumer(ILogger<OrderCreatedConsumer> logger)
```

```
    {
```

```
        _logger = logger;
```

```
        _consumerConfig = new ConsumerConfig
```

```

{
    BootstrapServers = "localhost:9092",
    GroupId = "order-processing-group",
    AutoOffsetReset = AutoOffsetReset.Earliest,
    EnableAutoCommit = true
};
}

```

protected override async Task ExecuteAsync(CancellationToken stoppingToken)

```

{
    await Task.Yield();

    using var consumer = new ConsumerBuilder<string, string>(_consumerConfig).Build();
    consumer.Subscribe("order-created");

```

```

        _logger.LogInformation("🚀 Consumer do Kafka iniciado. Escutando o tópico 'order-
created'...");

```

```

try
{
    while (!stoppingToken.IsCancellationRequested)
    {
        try
        {
            var consumeResult = consumer.Consume(stoppingToken);

            if (consumeResult?.Message != null)
            {

```

```

        var orderJson = consumeResult.Message.Value;

        _logger.LogInformation(" 📄 [Mensagem Recebida] Tópico: {Topic} | Chave: {Key}",
            consumeResult.Topic, consumeResult.Message.Key);

        ProcessOrder(orderJson);
    }
}

catch (ConsumeException ex)
{
    _logger.LogError(ex, " ❌ Erro ao consumir mensagem do Kafka.");
}

}

}

catch (OperationCanceledException)
{
    consumer.Close();

    _logger.LogInformation(" 🟢 Consumer do Kafka finalizado.");
}
}

```

```

private void ProcessOrder(string orderJson)
{
    try
    {
        using var document = JsonDocument.Parse(orderJson);

        var root = document.RootElement;

        var id = root.GetProperty("id").GetString();
    }
}

```

```

var customerName = root.GetProperty("customerName").GetString();

var amount = root.GetProperty("amount").GetDecimal();


_logger.LogInformation(" ⚡ [Processando Pedido] ID: {Id} | Cliente: {Customer} | Valor: R$
{Amount}",

    id, customerName, amount);
}

catch (Exception ex)
{
    _logger.LogError(ex, "Erro ao desserializar JSON do pedido.");
}
}
}

```

B. Configuração Principal (Program.cs)

Código completo registrando o **Producer**, **Consumer** e os **Endpoints**:

```

using System.ComponentModel.DataAnnotations;

using System.Text.Json;

using Confluent.Kafka;

var builder = WebApplication.CreateBuilder(args);

// 1. Configura e registra o Producer do Kafka
var producerConfig = new ProducerConfig
{
    BootstrapServers = "localhost:9092"
};

```

```
builder.Services.AddSingleton<IProducer<string, string>>(sp =>
{
    return new ProducerBuilder<string, string>(producerConfig).Build();
});
```

// 2. Registra o Consumer para rodar em background

```
builder.Services.AddHostedService<OrderCreatedConsumer>();
```

```
var app = builder.Build();
```

```
app.UseHttpsRedirection();
```

```
var ordersDatabase = new List<Order>();
```

// Endpoint POST - Producer

```
app.MapPost("/api/orders", async (CreateOrderRequest request, IProducer<string, string>
producer) =>
```

```
{
    if (string.IsNullOrEmpty(request.CustomerName) || request.Amount <= 0)
    {
        return Results.BadRequest(new { Message = "Dados do pedido inválidos." });
    }
}
```

```
var order = new Order(
    Id: Guid.NewGuid(),
    CustomerName: request.CustomerName,
    Amount: request.Amount,
```

```

        CreatedAt: DateTime.UtcNow,
        Status: "Created"
    );

    ordersDatabase.Add(order);

    var messageJson = JsonSerializer.Serialize(order);

    var kafkaMessage = new Message<string, string>
    {
        Key = order.Id.ToString(),
        Value = messageJson
    };

    // Publica no Kafka no tópico 'order-created'
    await producer.ProduceAsync("order-created", kafkaMessage);

    return Results.Created($"api/orders/{order.Id}", order);
});

// Endpoint GET
app.MapGet("/api/orders", () => Results.Ok(ordersDatabase));

app.Run();

// DTOs
public record CreateOrderRequest(
    [Required] string CustomerName,

```

[Required] decimal Amount

);

public record Order(

Guid Id,

string CustomerName,

decimal Amount,

DateTime CreatedAt,

string Status

);